buildings

MDPI

Article

LSH-YOLO: A Lightweight Algorithm for Helmet-Wear Detection

Zhao Liu, Fuwei Wang *, Weimin Wang, Shenyi Cao, Xinhao Gao and Mingxin Chen

School of Artificial Intelligence and Software, Liaoning Petrochemical University, No. 1 West Dandong Road, Wanghua District, Fushun 113001, China; liuzhao@stu.lnpu.edu.cn (Z.L.)
* Correspondence: wangfw4048@lnpu.edu.cn

Abstract

This work addresses the high computational cost and excessive parameter count associated with existing helmet-wearing detection models in complex construction scenarios. This paper proposes a lightweight helmet detection model, LSH-YOLO (Lightweight Safety Helmet) based on improvements to YOLOv8. First, the KernelWarehouse (KW) dynamic convolution is introduced to replace the standard convolution in the backbone and bottleneck structures. KW dynamically adjusts convolution kernels based on input features, thereby enhancing feature extraction and reducing redundant computation. Based on this, an improved C2f-KW module is proposed to further strengthen feature representation and lower computational complexity. Second, a lightweight detection head, SCDH (Shared Convolutional Detection Head), is designed to replace the original YOLOv8 Detect head. This modification maintains detection accuracy while further reducing both computational cost and parameter count. Finally, the Wise-IoU loss function is introduced to further enhance detection accuracy. Experimental results show that LSH-YOLO increases mAP50 by 0.6%, reaching 92.9%, while reducing computational cost by 63% and parameter count by 19%. Compared to YOLOv8n, LSH-YOLO demonstrates clear advantages in computational efficiency and detection performance, significantly lowering hardware resource requirements. These improvements make the model highly suitable for deployment in resource-constrained environments for real-time intelligent monitoring, thereby advancing the fields of industrial edge computing and intelligent safety surveillance.

Keywords: helmet wear detection; lightweight; YOLOv8; loss function; detection head

check for updates

Academic Editors: Difei Wu, Zheng Tong, Zhen Liu and Jinhuan Shan

Received: 1 July 2025
Revised: 19 July 2025
Accepted: 2 August 2025
Published: 18 August 2025

Citation: Liu, Z.; Wang, F.; Wang, W.; Cao, S.; Gao, X.; Chen, M. LSH-YOLO: A Lightweight Algorithm for Helmet-Wear Detection. *Buildings* **2025**, *15*, 2918. https://doi.org/10.3390/buildings15162918

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In construction sites, factory operations, and other industrial scenarios, the wearing of safety helmets is closely related to the life safety of workers. Traditional image detection methods typically rely on manually engineered features for object recognition [1]. For example, Park et al. [2] employed Histogram of Oriented Gradient (HOG) features combined with Support Vector Machine (SVM) classifiers for helmet detection. However, these methods require the extraction of a large number of feature samples to achieve effective recognition, resulting in substantial time and labor costs, and often suffer from poor robustness, limiting their applicability in practical environments. To improve supervision efficiency, helmet detection techniques based on computer vision have been widely adopted [3]. Current mainstream object detection algorithms can be broadly classified into two categories. The first is two-stage algorithms, which include methods such as R-CNN [4] and Faster R-CNN [5]. For instance, Fang et al. [6] applied Faster R-CNN to detect helmets

in scenes with far-field targets and complex construction backgrounds, achieving excellent detection results on their collected dataset. The second category is single-stage algorithms, such as SSD [7] and the YOLO series [8–10]. The core idea of this type of algorithm is to input an image into the model and directly predict the target's location and class from the raw image. Although the above detection models improve detection accuracy, they are often accompanied by high computational complexity and a large number of parameters. These issues result in significant hardware resource requirements, thereby limiting their deployment and application on resource-constrained devices. To address these challenges, many researchers have proposed improvements to network architectures.

He et al. [11] proposed an improved lightweight helmet detection algorithm, named YOLO-M3C. The algorithm replaces the YOLOv5s backbone network with MobileNetV3, reducing the model size from 13.7 MB to 10.2 MB. However, the backbone structure remains overly complex, and its feature extraction capability is still limited. Han et al. [12] proposed GhostNet, a lightweight end-to-end convolutional neural network used as the backbone. It incorporates multi-scale segmentation and feature fusion to improve detection accuracy. Despite these optimizations, the model still contains 11.2 million parameters. However, the number of parameters remains relatively high, posing challenges for deployment on extremely resource-constrained devices. Xing et al. [13] introduced a P2 detection layer and incorporated the lightweight Ghost module into YOLOv8, effectively reducing the computational cost to 4.2 GFLOPs. However, the detection head remained unoptimized, thereby limiting the overall lightweighting effect of the model. Fan et al. [14] optimized YOLOv8n by developing the C2f-GhostDynamicConv module combined with the asymmetric lightweight detection head LADH-Head, reducing the computation to 3.7 GFLOPs. However, even after our improvements, the model's computational cost remains approximately 20% higher. Although the aforementioned improvements have reduced model complexity and enabled deployment on low-power devices, they are still not optimal. Therefore, this paper proposes a lightweight helmet detection model, LSH-YOLO, based on an improved YOLOv8n. The model reduces computational cost to 3.0 GFLOPs while maintaining detection accuracy, making it more suitable for deployment in resource-constrained environments.

Although YOLO series models have achieved promising results in helmet-wearing detection, several technical bottlenecks remain in practical applications:

(1) High computational complexity and large parameter count: current mainstream methods, while improving detection accuracy, significantly increase computational burden and storage overhead, limiting deployment on resource-constrained devices (e.g., Jetson, mobile devices);
(2) Inconsistent detection performance across different target scales: when detecting small-scale or densely distributed targets (e.g., helmets worn by distant workers), conventional detection heads exhibit poor feature adaptation, negatively impacting overall accuracy;
(3) Insufficient loss function optimization for low- and medium-quality samples: the original IoU-based loss function suffers from an unreasonable gradient distribution for medium-quality anchor boxes, leading to inadequate focus on important samples during training, thus affecting the model's stability and generalization ability. To address these challenges, this study proposes an improved lightweight helmet detection model, LSH-YOLO, based on YOLOv8. The main contributions are as follows:

    (1) KWConv-based Backbone and Bottleneck Optimization: Ordinary convolutions in the backbone and bottleneck structures are replaced with KWConv, and an improved C2f-KW module is constructed, significantly reducing FLOPs and parameter counts while maintaining efficient feature extraction.

(2) Lightweight Detection Head Design: A lightweight detection head, SCDH (Shared Convolutional Detection Head), is proposed to enhance feature co-expression across different scales via a feature-sharing mechanism, effectively reducing redundant computations.

(3) Introduction of Wise-IoU v3 for Improved Localization: Wise-IoU v3 is adopted as the localization loss function, employing a non-monotonic focusing factor to adjust the gradient distribution. This encourages the model to focus more on medium-quality anchor boxes during training, enhancing stability and improving detection performance, especially for occluded targets.

(4) Experimental Validation: Experiments conducted on a construction site helmet detection dataset show that LSH-YOLO improves mAP50 by 0.6% to 92.9%, while reducing computational cost by 63% and parameter count by 19%, demonstrating both the effectiveness of the proposed method and its potential for practical deployment.

2. Analysis of the YOLOv8 Algorithm

YOLOv8, developed by Ultralytics, is a recent model in the YOLO (You Only Look Once) object detection series. It builds upon YOLOv4 [15] and YOLOv5 [16], offering improved detection accuracy and efficiency, and integrates advanced deep learning capabilities such as object detection, segmentation, pose estimation, tracking, and classification. Additionally, YOLOv8 provides models in various sizes, enabling flexible deployment across devices with different computational capacities. The overall architecture of YOLOv8 is illustrated in Figure 1.

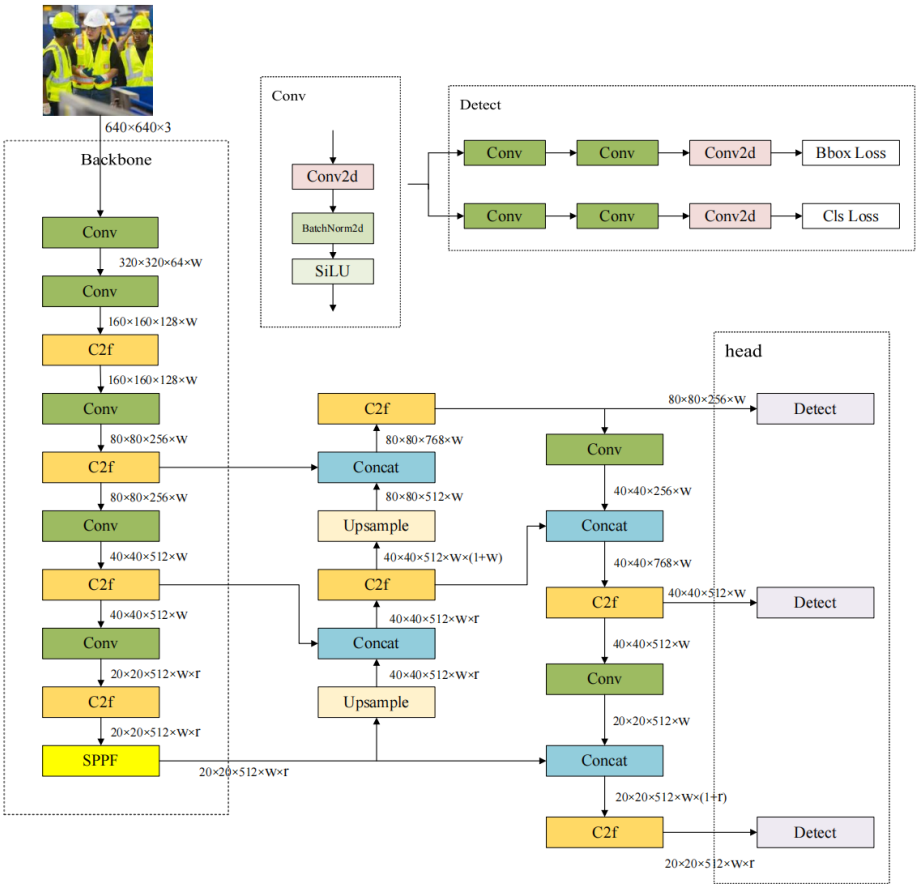


Figure 1. The architecture of the proposed YOLOv8 model.

YOLOv8 incorporates a more efficient backbone network and a dynamically adjustable receptive field design. It eliminates traditional anchor boxes in favor of an adaptive predic-

tion mechanism and employs decoupled detection heads to enhance both classification and localization accuracy. In addition, YOLOv8 enhances the loss function and label assignment strategy to support automatic mixed precision (AMP) training, which significantly accelerates training and reduces GPU memory usage. Furthermore, the model's generalization capability is enhanced by optimizing data augmentation techniques such as Mosaic [17]. However, the algorithm still exhibits limitations in complex scene recognition, including a restricted detection range, vulnerability to background interference, and reduced detection accuracy [18].

3. Innovations and Improvements of the Algorithm

To address the issues of excessive parameter count and high computational complexity in helmet detection algorithms, which impose stringent requirements on deployment hardware, this study proposes a series of model optimizations based on YOLOv8. The architecture of YOLOv8 is optimized in this study, with a focus on the convolutional layers (Conv), the C2f module, and the head layer. The standard convolutional layers (Conv) and the C2f module in the backbone of YOLOv8 are replaced with KWConv and the newly proposed C2f-KW module, respectively. At the head of the model, a lightweight detection head named SCDH is designed based on the concept of shared convolution to replace the original Detect head. Additionally, the normalization layer within the convolution blocks is replaced with group normalization (GN), which better suits lightweight architectures and helps maintain detection accuracy. Finally, to enhance the model's capability in image processing, the Wise-IoU loss function—featuring an intelligent gradient gain allocation strategy—is introduced to improve bounding box regression, significantly enhancing localization accuracy and detection performance in occluded and densely populated scenes. These enhancements effectively reduce the model's parameter count and computational load, while improving inference accuracy and ensuring efficient performance across devices with varying computational capabilities. The structure of the improved model is shown in Figure 2.

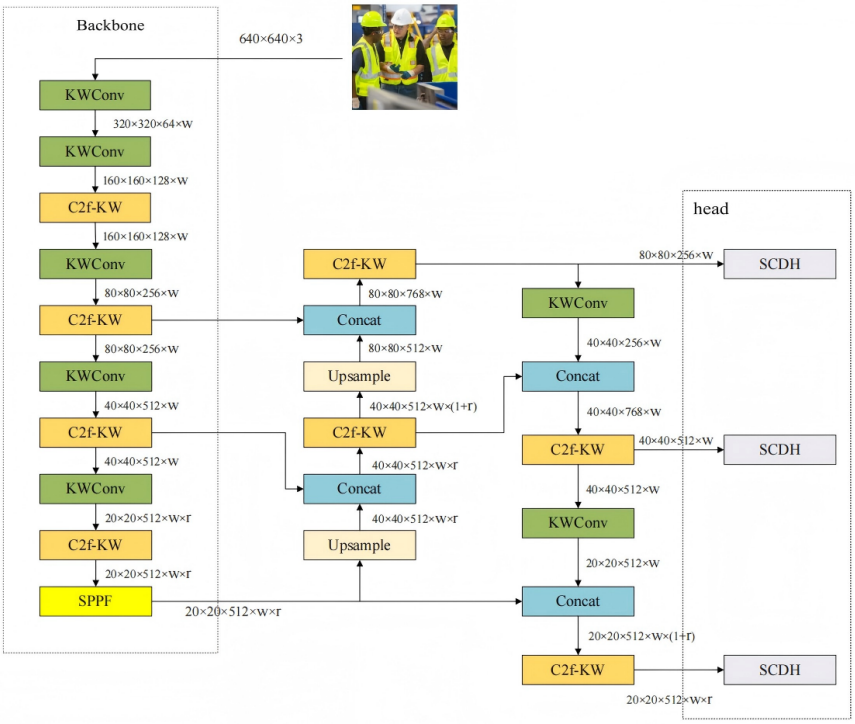


Figure 2. The architecture of the proposed LSH-YOLO model.

3.1. KWConv Module

Although convolutional neural networks (CNNs) in the YOLO series exhibit strong feature extraction capabilities, they also present notable limitations, particularly in computational efficiency and detection accuracy. First, the number of parameters and the computational cost of CNNs increase exponentially with the number of input channels and convolutional kernels, leading to high overall model complexity. This makes them unsuitable for deployment on resource-constrained platforms such as edge devices. Second, CNNs have fixed receptive fields and limited capability in capturing fine-grained details of small objects, making them prone to missed detections in densely packed or small-object scenarios. Moreover, in complex environments involving lighting variation, occlusion, or image blurring, standard convolution lacks adaptability, making it difficult to extract robust features effectively, which degrades overall model performance.

To meet the deployment requirements in complex scenarios and resource-constrained environments, the standard convolution is replaced with KWConv, inspired by the Kernel-Warehouse (KW) [19]. The core concept of KWConv is to reformulate dynamic convolution by leveraging parameter dependencies within and across layers of the convolutional network. This approach aims to achieve a better trade-off between parameter efficiency and representational capacity. As shown in Figure 3, the KW structure comprises three interdependent components: kernel partitioning, repository sharing, and the Channel-Aware Fusion (CAF) attention mechanism.

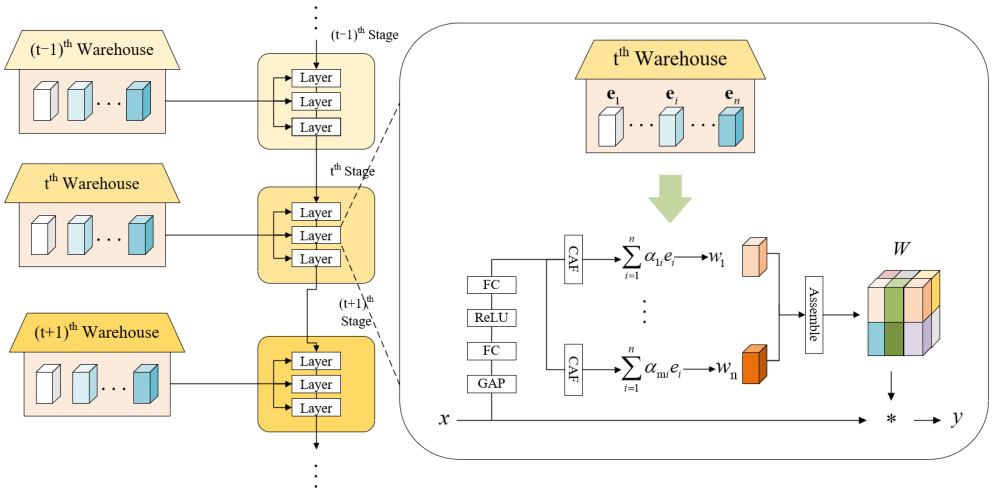


Figure 3. The architecture of KernelWarehouse.

Kernel Partitioning and Repository Sharing: The core concept of kernel partitioning is to leverage parameter dependencies within the same convolutional layer to reduce the dimensionality of the convolution kernel. Specifically, in a standard convolutional layer, the static convolution kernel is divided into several disjoint sub-kernels, termed kernel cells, which share the same dimensionality and are partitioned based on spatial and channel dimensions. The kernel partition can be formally defined as:

$$W = w_1 \cup \ldots \cup w_m \forall i, j \in \{1, 2, \ldots, m\},\ i \neq j \Rightarrow w_i \cap w_j = \varnothing \tag{1}$$

After kernel partitioning, the kernel unit $w_1, \ldots, w_m$ is treated as a localized kernel. We define a "warehouse" $\mathbf{E} = \{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n\}$, which contains m kernel units, as $\mathbf{E} = \{\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n\}$, where $\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n$ are the same dimensions as $w_1, \ldots, w_m$. A set of core units is represented as a linear combination of elements in the core warehouse:

$$w_i = \alpha_{i1}\mathbf{e}_1 + \cdots + \alpha_{in}\mathbf{e}_n, \quad \text{and} \quad i \in \{1, \ldots, m\} \tag{2}$$

where $\alpha_{i1}, \ldots, \alpha_{in}$ denotes the attention coefficient produced by the attention module. Ultimately, the static convolutional kernel W in the conventional convolutional layer is substituted by its corresponding linear mixture of m.

n denotes the number of basis kernel units stored in the core warehouse E, while m represents the number of sub-kernels obtained after partitioning the original convolutional kernel. Typically, $n < m$ is set to enable parameter sharing and reduce redundancy.

Warehouse sharing enhances computational efficiency and reduces model parameters by allowing convolutional kernels to share parameters across different kernels. By organizing multiple kernels into a unified "repository" and applying linear combinations of these kernel units, the model can flexibly adapt to varying input features, thereby minimizing redundant computation while maintaining or even enhancing feature extraction capability. This parameter-sharing mechanism strengthens the model's representational power, particularly in complex tasks, improves generalization performance, and further alleviates computational overhead.

3.2. C2f-KW Module

The original YOLOv8 model contains a substantial number of C2f modules, which leads to increased parameter counts and limited feature learning capacity. In helmet detection tasks, targets are often occluded and frequently consist of small objects. Under such conditions, the original C2f module lacks sufficient capacity for effective feature extraction and multi-scale information fusion, which negatively impacts detection performance across diverse environments. Moreover, in complex environments, targets may be affected by lighting variations, occlusions, or background interference, leading to unclear texture details, blurred edges, and significant information loss, thereby weakening the model's feature representation capability. To address the aforementioned issues, the Bottleneck module in YOLOv8n is optimized by replacing the standard convolution with KWConv based on the concept introduced in KernelWarehouse (KW). As illustrated in Figure 4, KWConv dynamically adjusts the kernel sizes across different channels to enhance the adaptability of the receptive field. Subsequently, the SiLU (Sigmoid Linear Unit) activation function is applied to introduce non-linearity and strengthen the model's feature representation capability. Finally, the Bottleneck-KW module is built based on this structure and used to replace the original Bottleneck module in the C2f block, forming the improved C2f-KW module. This enhancement effectively strengthens feature extraction while reducing computational cost. The structure of the improved C2f-KW module is shown in Figure 5.

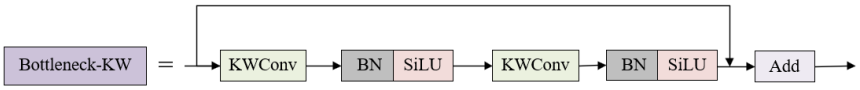


Figure 4. The Bottleneck-KW module.

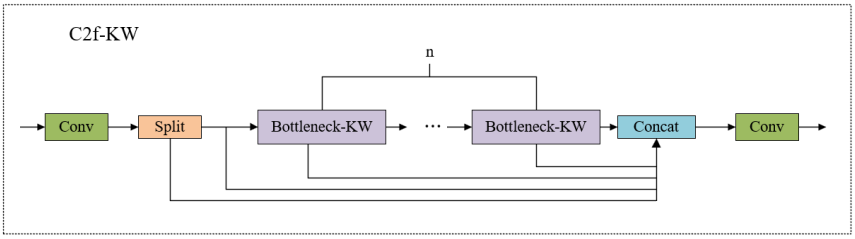


Figure 5. The C2f-KW structure.

The C2f-KW module significantly reduces the model's computational complexity by introducing the concept of KW dynamic convolution, which partitions the convolution kernel into multiple local kernel units with shared parameters. Each local kernel unit is

dynamically adjusted based on attention coefficients generated from the input, thereby enhancing the model's flexibility and adaptability. This module not only reduces computational burden but also enhances feature extraction capability, particularly in complex environments and scenarios involving small targets. Overall, the C2f-KW module improves both the efficiency and detection performance of the model, making it more suitable for deployment on devices with limited computational resources.

3.3. Lightweight Detection Head: SCDH Module

YOLOv8 employs a decoupled detection head that separates classification, regression, and confidence prediction branches to enhance detection accuracy. Specifically, the regression branch predicts the bounding box coordinates, the classification branch determines the object category, and the confidence branch assesses the presence of an object. Compared with traditional coupled detection heads, the decoupled architecture reduces task interference and improves overall detection performance. However, this structure also increases computational complexity and the number of parameters. To address these limitations, we propose an improved detection head by introducing a Shared Convolutional Detection Head (SCDH), which effectively enhances performance while reducing computational overhead. The architecture of SCDH is illustrated in Figure 6.

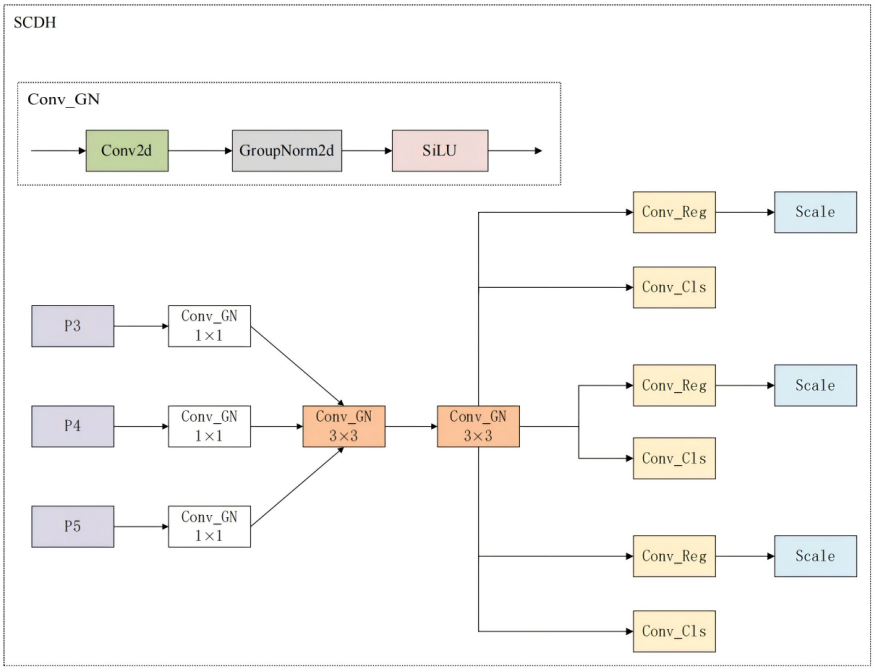


Figure 6. The architecture of the SCDH module.

The detection head receives feature inputs from the P3, P4, and P5 levels of the feature pyramid. It first employs a 1 × 1 shared convolution to facilitate channel-wise information interaction and extract features at different scales. Subsequently, two 3 × 3 shared convolutions are applied for feature aggregation, reducing redundant information and enhancing the learning and extraction of distinct features. Finally, the Scale layer performs feature scaling to further enhance adaptability across multiple scales, enabling the detection head to effectively process inputs with varying resolutions. For normalization, SCDH replaces traditional Batch Normalization (BN) [20] with the Conv_GN module, which adopts the more suitable Group Normalization (GN).

The advantages of this detection head are reflected in the following aspects:

(1) Normalization strategy optimization: Group Normalization (GN) is adopted to replace Batch Normalization (BN), addressing the instability of BN under small-batch training and eliminating dependence on batch size. This ensures stable detection performance under varying training conditions [21], particularly in small-batch scenarios.
(2) Shared convolution structure: The detection head employs a shared convolution strategy, where the output of the 1 × 1 Conv_GN is fed into the 3 × 3 Conv_GN, and the same convolution kernel is shared across all spatial locations for feature extraction. This approach enhances feature representation while significantly reducing computational complexity, thereby improving the overall efficiency of the detection head.
(3) Robust scale consistency and adaptability: By unifying the scale representation across different feature map layers using shared convolution and scale layers, the detection head achieves strong adaptability to both small and large targets.
(4) Decoupled detection branches: The detection head decouples classification and regression branches, enabling each task to learn optimal features independently. This reduces task interference and improves both localization and classification accuracy.

3.4. Loss Function: Wise-IoU

YOLOv8 adopts the CIoU loss as the bounding box localization loss function, which is an improvement over the GIoU and Distance DIoU losses. The CIoU loss considers the distance between the centroids of the predicted and ground-truth boxes, the overlap area, and the aspect ratio, thereby improving localization accuracy and accelerating convergence [22]. The mathematical formulation of the CIoU loss function is presented below:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \tag{3}$$

where IoU (Intersection over Union) represents the ratio of the overlapping area between the predicted box and the ground-truth box to their union. $\rho^2(\mathbf{b}, \mathbf{b}^{gt})$ represents the Euclidean distance between the centers of the predicted and ground-truth boxes. c^2 Denotes the square of the diagonal length of the minimum enclosing box covering both the predicted and ground-truth boxes. a An adaptive weighting factor, defined as follows:

$$\alpha = \frac{v}{(1 - IoU) + v} \tag{4}$$

v denotes the aspect ratio consistency and is defined as follows:

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2 \tag{5}$$

where w and h represent the width and height of the predicted bounding box, and w^{gt} and h^{gt} represent the width and height of the ground truth box, respectively.

CIoU exhibits good stability by addressing the issue of drifting detection frames through the inclusion of centroid distance. Additionally, constraints are introduced to align the shape of the predicted frame with the ground truth, enhancing detection accuracy. This approach particularly enhances performance in small target detection. However, while CIoU introduces an aspect ratio penalty term, its impact is insufficient, particularly for extreme targets (e.g., occlusions). Furthermore, although CIoU enhances localization accuracy at low IoU values, the influence of the aspect ratio term tends to diminish at higher IoU levels. This imbalance may hinder model optimization and reduce the ability to effectively distinguish between closely spaced targets effectively [23]. To address the need for high accuracy in complex environments, we introduce the Wise-IoU loss function, which further optimizes IoU computation and enhances convergence speed and stability

compared to CIoU [24]. The Wise-IoU loss function has three versions, and in this study, we employ the Wise-IoU v3 version, with the following loss function:

$$\mathcal{L}_{WIoU-v3} = r \cdot \mathcal{L}_{WIoU-v1}, \quad r = \frac{\beta}{\delta \alpha^{\beta-\delta}} \tag{6}$$

$$\beta = \frac{L_{IoU}^{*}}{L_{IoU}} \tag{7}$$

where $\mathcal{L}_{WIoU-v1}$ represents the underlying Wise-IoU loss:

$$\mathcal{L}_{WIoU-v1} = R_{WIoU} \cdot L_{IoU} \tag{8}$$

$$R_{WIoU} = \exp\left(\frac{(x - x_{gt})^2 + (y - y_{gt})^2}{(W_g^2 + H_g^2)^{*}}\right) \tag{9}$$

R_{WIoU} represents the distance-attention mechanism, where W_g^2, H_g^2 denote the dimensions of the smallest enclosing box. An asterisk (*) indicates that this part is separated in the computational graph, preventing gradient backpropagation. WiFse-IoU v3 adjusts the gradient distribution using a non-monotonic focusing factor r, which helps concentrate the training on medium-quality anchor boxes. Smaller gradients are assigned to low-quality boxes (β large) to mitigate the impact of harmful gradients, while high-quality boxes (β small) also receive weakened gradients to prevent wasting computational resources. The hyperparameters δ and α regulate the gradient distribution curve, contributing to more stable and efficient optimization. This enhances the model's generalization ability and improves detection performance for occluded targets. In this experiment, the application of the Wise-IoU v3 loss function directs the model's focus toward medium-quality boxes during training, reduces computational resource waste, enhances model stability, and accelerates convergence. The experimental results demonstrate an improvement in the model's mAP50 after employing Wise-IoU v3, highlighting superior performance in detecting small and occluded targets, with further enhancement in detection accuracy.

As shown in Figure 7, the loss curves of YOLOv8n and LSH-YOLO are compared. LSH-YOLO exhibits a faster decline and better convergence across all three loss components during training, indicating a stronger fitting capability. On the validation set, the loss values of LSH-YOLO stabilize after the 20th epoch without significant fluctuations or continuous increases, suggesting the absence of overfitting. Moreover, its overall validation loss remains consistently lower than that of YOLOv8n, reflecting superior generalization ability. Notably, LSH-YOLO consistently achieves lower errors in box_loss and dfl_loss, indicating improved accuracy in bounding box localization. In addition, the cls_loss of LSH-YOLO exhibits slight fluctuations during early training but converges rapidly in later stages, ultimately outperforming YOLOv8n. This suggests that its classification performance is more stable and reliable under complex conditions. Overall, LSH-YOLO outperforms YOLOv8n in terms of training stability and validation performance. This demonstrates that the lightweight improvements do not compromise accuracy, highlighting the model's strong potential for practical deployment.

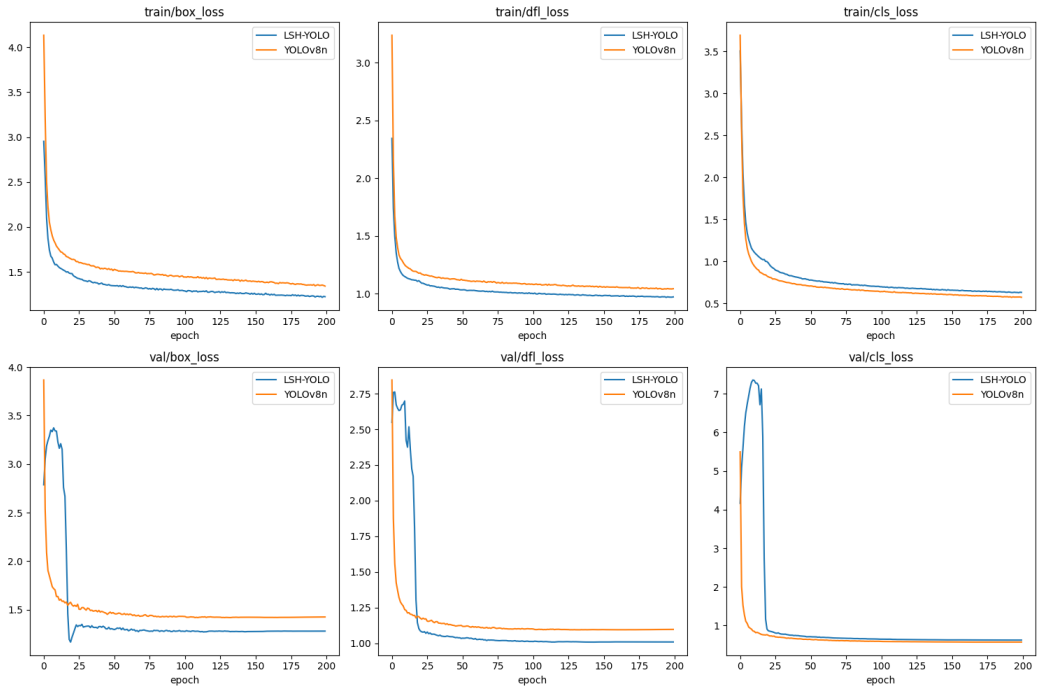


Figure 7. Comparison of training and validation loss for YOLOv8 and LSH-YOLO.

4. Experiments and Analysis

4.1. Experimental Environment

The hardware environment used in the experiment and the hyperparameter settings for training are presented in Table 1.

Table 1. Details of the experimental environment and hyperparameters.

Name	Parameter
Operating System	Windows11
Programming Language	Python3.9
Cpu	Intel Corei5-12400F
Gpu	RTX 4060
Memory	32 G
Algorithm Framework	Pytorch2.2.2
Epoch (hyperparameter)	200
Batch_size (hyperparameter)	16
lr0 (hyperparameter)	0.005
Optimizer (hyperparameter)	SGD
Imges_Size (hyperparameter)	640

4.2. Dataset

4.2.1. Dataset Introduction and Splitting

This study utilizes the publicly available SHWD helmet-wearing detection dataset, comprising 7581 labeled images with rich and diverse content. The dataset encompasses multiple indoor and outdoor construction scenarios, such as aerial work, ground construction, and mechanical operations. Workers are captured in various postures from multiple viewpoints and distances—including front, side, and back views—as well as both dynamic and static poses, reflecting the complexity and diversity of the scenes. The dataset is divided into training, validation, and test sets in a 7:2:1 ratio. The dataset includes helmet-wearing targets of various scales—large, medium, and small—captured in diverse scenes. The number of instances per category is shown in the histogram in Figure 8a, which

presents 9044 helmet-wearing instances and 111,514 non-helmet (normal head) instances. The categories include helmet wearers (helmet) and non-helmet wearers (head). Figure 8b illustrates the bounding box distribution across the entire dataset. In Figure 8c, the x and y axes represent the center coordinates of the target boxes; darker colors indicate higher density of box centers. Figure 8d displays the height-to-width ratio of targets relative to the entire image. Darker colors indicate a higher number of samples. Figure 9 displays example images from the SHWD dataset.

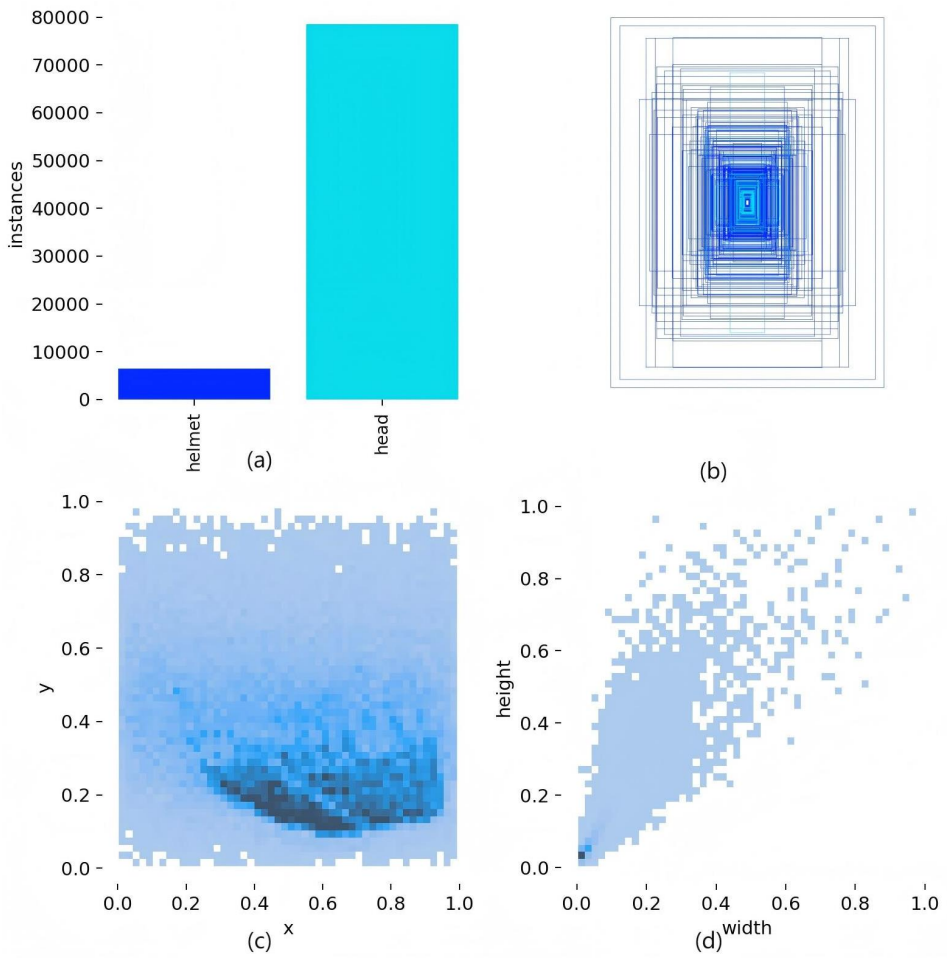


Figure 8. Visualization of the annotated training data. (**a**) Number and categories of helmet-wearing and non-helmet instances; (**b**) overall bounding box distribution; (**c**) distribution of bounding box center coordinates; (**d**) height-to-width ratio distribution of targets.



(**a**) (**b**) (**c**)

Figure 9. Sample images from the SHWD dataset. (**a**–**c**) Partial images showing different scenarios and variations in helmet-wearing and non-helmet instances.

4.2.2. Dataset Processing

During training, to further enhance the robustness and diversity of the dataset, a series of data augmentation techniques were applied to the original images, including noise addition, random horizontal and vertical flipping, and brightness adjustment. These varied transformations effectively expand the training samples, enabling the model to adapt to more complex and dynamic real-world scenarios. This approach not only enriches data representation but also strengthens the model's robustness against noise, lighting variations, and changes in image orientation, thereby significantly improving its generalization and stability. By simulating diverse real-world conditions, the model maintains high recognition performance across different input variations.

4.3. Evaluation Metrics

Evaluating the performance of YOLO series algorithms in object detection tasks is essential. Common evaluation metrics for object detection include model accuracy, model size, and resource consumption, among others. Common evaluation metrics for object detection include model accuracy, model size, and resource consumption, among others. FLOPs measures the computational complexity of the model, Params represents the model's storage requirements, and mAP quantifies the mean accuracy across all categories. Lower FLOPs and Params indicate reduced computational burden and memory usage, enabling the model to operate efficiently on a wide range of hardware platforms and facilitating deployment in resource-constrained environments.

In the helmet-wearing detection task, the Precision metric indicates the proportion of correctly predicted helmet-wearing instances among all samples predicted as wearing a helmet. The formula is as follows:

$$\mathrm{P} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{10}$$

In the helmet-wearing detection task, the Recall metric represents the proportion of correctly predicted helmet-wearing samples to the total number of actual helmet-wearing instances, calculated using the following formula:

$$\mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{11}$$

The values of recall range from 1 to 0 (Recall $\in [0, 1]$), with 1 indicating that the model found all positive class samples and 0 indicating that the model did not find positive class samples.

Average Precision (AP) measures the average precision for a single category, while mean Average Precision (mAP) represents the average of AP values across all categories. In this study, mAP50 is used for evaluation, where the IoU threshold is set to 0.5, and the mAP is computed by first averaging AP across categories. The mAP is calculated as follows:

$$\mathrm{mAP} = \frac{1}{N} \sum_{i=1}^{N} \mathrm{AP}_i \tag{12}$$

where N denotes the total number of categories, and AP_i represents the average precision of the category. The formula is given as follows:

$$\mathrm{AP} = \int_0^1 \mathrm{Precision}(r)\, dr \tag{13}$$

The discrete calculation formula of AP is as follows:

$$\mathrm{AP} = \sum_{k} (R_k - R_{k-1}) \cdot P_k \tag{14}$$

where R_k represents the recall at the k point, P_k denotes the interpolated precision at the k point, and $R_k - R_{k-1}$ signifies the increment in recall.

4.4. Ablation Study

The experiments were conducted using the control variable method to evaluate the optimization effects of the improvements made to each module of the YOLOv8n algorithm on the publicly available SHWD dataset. To ensure a fair comparison, the experimental environment and parameter settings were kept consistent, and ablation experiments were designed, with ✓ indicating the corresponding method used, × indicates the method was not used.as shown in Table 2. Experiment 1 presents the results for the benchmark YOLOv8n model, which requires 8.1 G of computation, making it difficult to meet the lightweight and deployment requirements for edge devices. Experiment 2 optimizes the regular convolution and enhances the C2f module using KW, reducing the computational volume by 44% compared to the original YOLOv8n, with no decrease in accuracy and only a 0.2% reduction in recall. However, the model size increases by 0.23 MB to accommodate the memory constraints of devices, while simultaneously reducing both computational and parametric volume. Experiment 5 improves the model's detection head (SCDH module) by incorporating KW, resulting in a 0.5% increase in accuracy, while further reducing both computation and parameters. Finally, Experiment 6 introduces the Wise-IoU loss function, which improves both mAP50 and recall, without changing the model size.

The LSH-YOLO algorithm is improved by incorporating the KW, SCDH module, and Wise-IoU loss function. After 200 rounds of iterative training, the computational cost is reduced by 63%, the parameter count is decreased by 19%, mAP50 is improved by 0.6%, reaching 92.9%, and the model size is reduced accordingly. The improved model demonstrates excellent performance in both model lightweighting and detection accuracy, making it well-suited for lightweight helmet target detection on platforms with limited computational resources.

Table 2. Results of the ablation study.

ID	KW	SCDH	Wise-IoU	FLOPs (G)	$mAP_{0.5}$ (%)	$mAP_{0.5:0.95}$ (%)	P (%)	R (%)	Model Size (M)
1	×	×	×	8.1	92.3	58.1	92.3	86.8	5.95
2	✓	×	×	4.5	92.1	58.0	92.3	86.6	6.22
3	×	✓	×	6.5	92.7	58.5	92.3	87.1	4.70
4	×	×	✓	8.1	92.4	58.1	92.5	86.6	5.95
5	✓	✓	×	3.0	92.4	57.7	92.8	86.1	4.97
6	✓	✓	✓	3.0	92.9	57.7	91.7	87.4	4.97

According to the training curves shown in Figure 10, the improved LSH-YOLO model demonstrates a faster convergence rate and a smoother upward trend in precision, recall, and mAP, with an overall trajectory that ascends more rapidly and steadily. This suggests that the model is able to quickly extract effective features during the early training stages and maintain more stable gradient updates throughout subsequent training, resulting in a more robust performance improvement process. This demonstrates that the proposed shared convolutional structure effectively reduces redundant computation and enhances the consistent representation of multi-scale features. Meanwhile, the incorporation of the Wise-IoU loss function alleviates the adverse impact of extreme samples during training, thereby further improving the model's robustness and convergence.

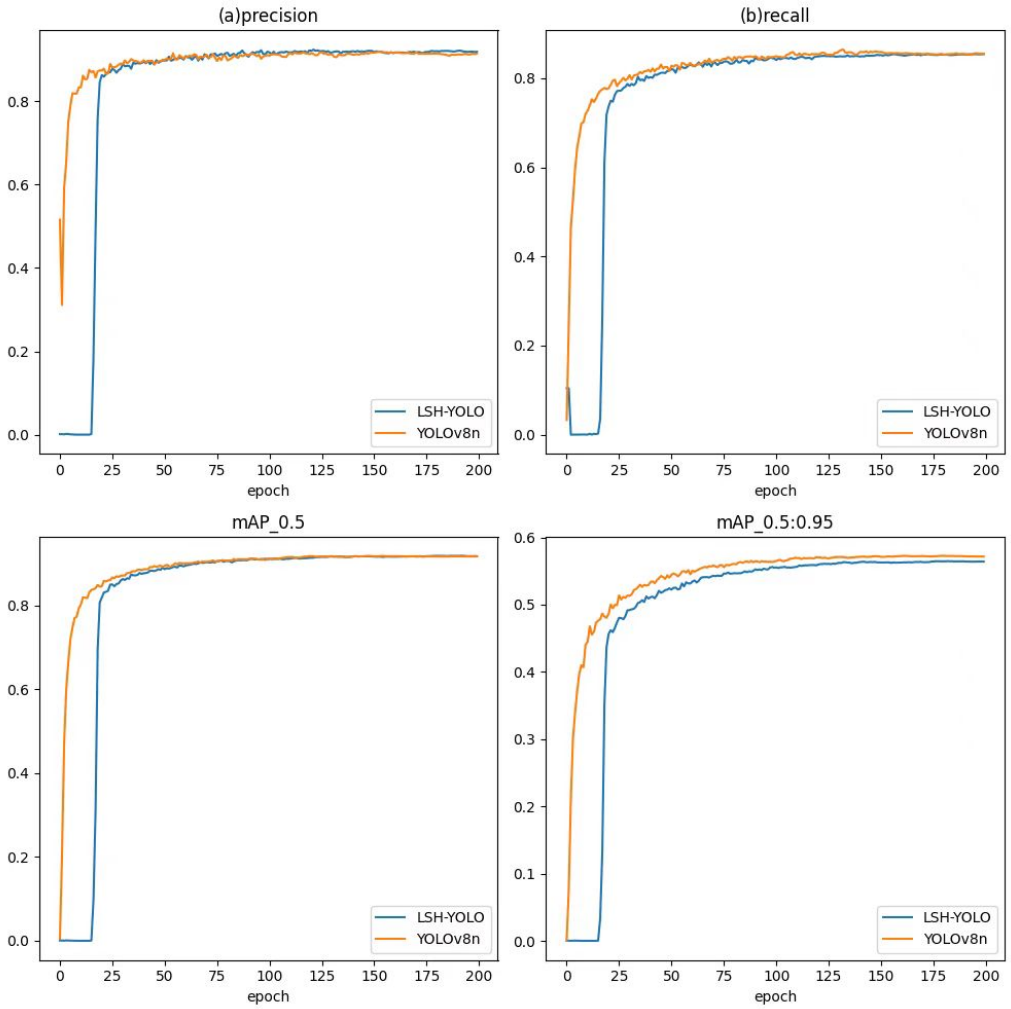


Figure 10. The R and AP curves of YOLOv8n and LSH-YOLO.

4.5. Comparative Experiment

To demonstrate the superiority of the improved LSH-YOLO model, comparative experiments were conducted with other mainstream models, while controlling for external variables such as hardware and software environments. The evaluation metrics include key indicators such as computational cost, number of parameters, accuracy, recall, mAP50, and model size. The experimental results are presented in Table 3.

Analysis of the data reveals that, for helmet detection, this algorithm not only outperforms the second-stage model, Faster R-CNN, in terms of mAP50, but also demonstrates significantly lower computational cost. With a computational cost of 190 G, Faster R-CNN clearly fails to meet lightweight requirements, making it unsuitable for deployment on resource-constrained devices. To meet the requirement for lower computational cost, we compare the algorithm with the YOLOv8n, a model from the YOLO series, which has a similar computational cost. It can be concluded that LSH-YOLO demonstrates significant superiority at a similar computational cost, achieving a generally higher mAP50 than other algorithms. One of these algorithms, YOLOv5s, achieves an mAP50 of 93.2%, which is 0.9% higher than the benchmark YOLOv8n and 0.2% higher than the improved LSH-YOLO model. However, YOLOv5s requires 15.8 G of computation, more than four times the computational cost of the improved LSH-YOLO model. When compared to the GE-YOLO algorithm proposed by Zhang et al [25] under the same lightweight constraints, the proposed algorithm reduces computational cost by 48%, increases mAP50 by 0.4%, and decreases the number of parameters. Overall, the improved LSH-YOLO algorithm

effectively meets the requirements for deployment on resource-constrained platforms. This clearly demonstrates the superiority of the proposed algorithm.

In conclusion, the improved LSH-YOLO algorithm effectively meets the requirements for deployment or operation in resource-constrained environments, showcasing its superiority as demonstrated in this study.

Table 3. Results of the comparative experiment.

Model	FLOPs (G)	$mAP_{0.5}$ (%)	$mAP_{0.5:0.95}$ (%)	P (%)	R (%)	Model Size (M)
YOLOv5n	4.1	91.6	56.3	92.4	87.2	3.65
YOLOv5s	15.8	93.2	58.7	93.8	88.1	13.6
YOLOv7tiny	13.0	92.7	55.9	92.7	87.1	11.6
YOLOv8n	8.1	92.3	58.1	92.3	86.8	5.95
YOLOv9t	10.7	92.3	58.3	92.0	87.0	17.1
YOLOv10n	6.5	92.2	58.4	92.2	86.1	5.22
YOLOv11n	6.3	92.1	58.3	92.0	86.9	5.22
YOLOv12n	5.8	92.1	57.9	91.7	86.7	5.18
RT-DETR	56.9	93.4	59.2	93.9	88.7	38.5
Faster R-CNN	190	91.7	52.4	88.2	82.3	315
swintransformer	79.1	91.3	56.3	91.2	85.5	60.5
GE-YOLO	5.8	92.5	–	93.4	86.5	–
Ours	3.0	92.9	57.7	91.7	87.4	4.97

To further verify the adaptability and generalization ability of the model in diverse scenarios, this study introduces the open-source GDUT-HWD dataset as an external test set. Compared with the SHWD dataset used for training, GDUT-HWD differs significantly in terms of shooting angles, image resolutions, target scales, and background environments. It encompasses more complex construction scenes, thereby providing a more comprehensive evaluation of the model's performance in real-world applications. The original annotations in the GDUT-HWD dataset include five categories: red, yellow, white, blue, and none, corresponding to the presence of red, yellow, white, blue, or no helmet, respectively. To ensure consistency with the SHWD dataset categories, the GDUT-HWD dataset was filtered and relabeled, with all helmet-wearing categories merged into (helmet) and the unprotected head category labeled as (head).

4.6. Generalization Experiment

As shown in Table 4, YOLOv8n achieves a mAP@0.5 of 83.7 and a mAP@0.5:0.95 of 49.5 on the GDUT-HWD dataset, reflecting strong overall performance. In comparison, LSH-YOLO achieves a slightly lower mAP (83.4 at mAP@0.5 and 48.8 at mAP@0.5:0.95), but improves slightly in Precision, reaching 0.888. This suggests that LSH-YOLO is more accurate in identifying positive samples and exhibits stronger suppression of false positives. Overall, while significantly reducing computational complexity and parameter count, LSH-YOLO maintains comparable detection performance to YOLOv8n on heterogeneous datasets. This demonstrates its robust generalization capability and strong potential for real-world deployment in engineering scenarios.

Table 4. Results of the generalization experiment.

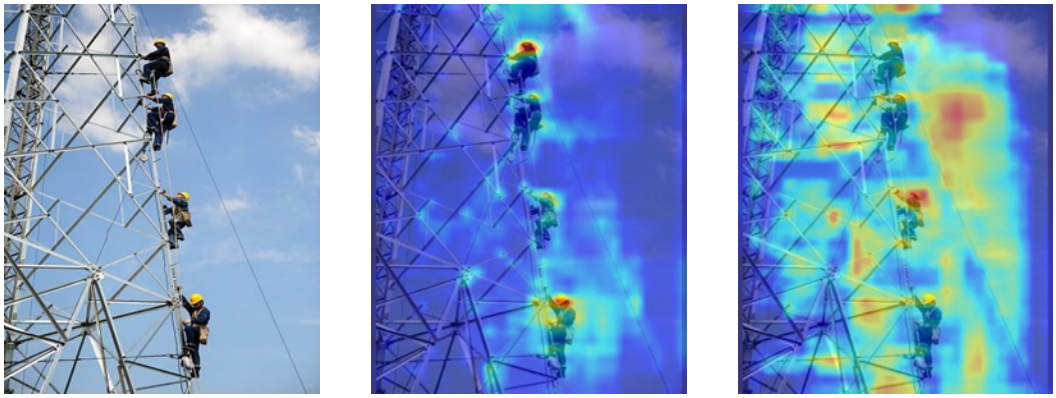
	FLOPs/G	$mAP_{0.5}$ (%)	$mAP_{0.5:0.95}$ (%)	P (%)	R (%)
YOLOv8n	8.1	83.7	49.5	88.4	75.5
LSH-YOLO	3.0	83.4	48.8	88.8	75.4

4.7. Experimental Results and Analysis

4.7.1. Heatmap

To provide a more intuitive visualization of the model's focus on the detected object, this paper employs the GradCAM method to generate heat maps for both models, illustrating the degree of attention given to the target features. Initially, images from the test set are selected for detection, generating the corresponding heat maps. As depicted in the Figure 11, the color gradient from blue (indicating low attention) to red (indicating high attention) represents an increasing level of focus, with the red regions more concentrated around the target area signifying greater attention to the detection target.

As shown in the heat maps of different models in Figure 11, the YOLOv8 model exhibits a stronger focus on the target region (indicated by a higher concentration of red), but also displays more irrelevant feature focus in non-target areas, suggesting the presence of extraneous feature extraction. In contrast, the improved detection model focuses more precisely on the target region, reducing the influence of irrelevant backgrounds and enhancing the effectiveness of feature extraction as well as detection robustness. This indicates that the improved model offers greater accuracy in feature representation and target localization, further enhancing detection performance.



(**a**) Original Image (**b**) YOLOv8n (**c**) LSH-YOLO

Figure 11. Heatmap comparison of performance across different models.

4.7.2. Visualization of Detection Results

To demonstrate the effectiveness of the improved LSH-YOLO model, three types of representative images from the test set were selected for detection. These include scenarios of no helmet being worn, dense occlusion, and false detection.

As shown in Figure 12, in the case of no helmet being worn, the experimental results demonstrate that the detection model accurately distinguishes between helmet-wearing and non-helmet-wearing scenarios. The model does not detect the absence of a helmet, and detection occurs only when the helmet is either worn or not worn. This characteristic aligns well with real-world applications, effectively reducing false detection rates while enabling accurate monitoring of the construction site, thereby enhancing the practicality and reliability of the detection system.

Compared with (a) in Figure 13, under dense small-target scenarios with visual interference, the LSH-YOLO model successfully detects a red helmet missed by the original model. This demonstrates that the improved model not only excels in detecting dense targets but also significantly reduces missed detections caused by occlusion.

Compared with (a1) and (a2) in Figure 14, the LSH-YOLO model demonstrates higher accuracy in identifying true targets under complex and low-light conditions, while effectively reducing missed detections.



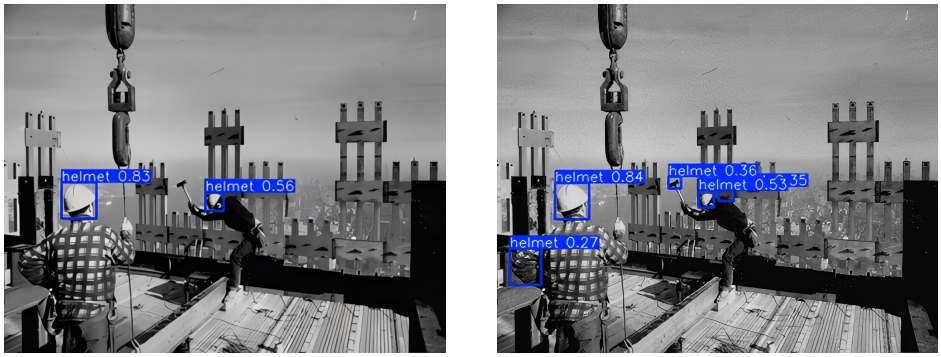
(**a**) Original Image (**b**) LSH-YOLO

Figure 12. Examples where the helmet is not worn.



(**a**) YOLOv8n (**b**) LSH-YOLO

Figure 13. Exclusion of missed detections due to occlusion in dense scenes.



(**a1**) (**b1**)



(**a2**) YOLOv8n (**b2**) LSH-YOLO

Figure 14. Example of false-positive detections being excluded, subfigures (**a1**,**a2**) and (**b1**,**b2**) show the detection results of YOLOv8n and LSH-YOLO, respectively.

5. Conclusions

This paper proposes a lightweight helmet detection algorithm, LSH-YOLO (Lightweight Safety Helmet) based on an improved YOLOv8n framework. By integrating KernelWarehouse (KW) modules into the backbone and neck, designing a lightweight shared convolutional detection head (SCDH), and employing the Wise-IoU loss function, LSH-YOLO significantly enhances detection accuracy and localization robustness while substantially reducing computational cost and parameter count. Experimental results demonstrate that LSH-YOLO achieves an excellent trade-off between model compactness and detection performance, reducing computational complexity by 63%, parameters by 19%, model size, and improving mAP@50 to 92.9%. This approach not only offers an efficient and cost-effective solution for helmet detection but also lays a foundation for deploying lightweight target detection models on resource-constrained devices. It is particularly suitable for edge computing environments demanding real-time processing, such as intelligent construction site monitoring, industrial automation inspection, and urban security surveillance, thereby exhibiting broad applicability. Despite these promising results, certain limitations remain. The model's robustness under highly complex backgrounds and severe occlusions requires further enhancement. Moreover, current experiments are limited to single-class detection, and generalization to multi-class recognition tasks remains to be validated. Future research will explore the model's adaptability to multi-category targets, its multi-task learning capabilities, and its performance in more complex real-world environments, aiming to advance edge intelligence applications in industrial scenarios.

Author Contributions: Z.L.: conceptualization, methodology, validation, formal analysis, writing—original draft preparation, and writing—review and editing. F.W.: writing—review and editing, and supervision. W.W.: methodology and project administration. S.C.: conceptualization, methodology, and validation. X.G.: methodology, formal analysis, and supervision. M.C.: methodology. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Rubaiyat, A.H.; Toma, T.T.; Kalantari-Khandani, M.; Rahman, S.A.; Chen, L.; Ye, Y.; Pan, C.S. Automatic detection of helmet uses for construction safety. In Proceedings of the 2016 IEEE/WIC/ACM International Conference on Web Intelligence Workshops (WIW), Omaha, NE, USA, 13–16 October 2016; pp. 135–142.
2. Park, M.W.; Elsafty, N.; Zhu, Z. Hardhat-wearing detection for enhancing on-site safety of construction workers. *J. Constr. Eng. Manag.* **2015**, *141*, 04015024. [CrossRef]
3. Cheng, L. A Highly robust helmet detection algorithm based on YOLO V8 and Transformer. *IEEE Access* **2024**, *12*, 130693–130705. [CrossRef]
4. Girshick, R.; Donahue, J.; Darrell, T.; Malik, J. Rich feature hierarchies for accurate object detection and semantic segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 23–28 June 2014; pp. 580–587.
5. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. *Adv. Neural Inf. Process. Syst.* **2015**, *28*, 91–99. [CrossRef] [PubMed]
6. Fang, Q.; Li, H.; Luo, X.; Ding, L.; Luo, H.; Rose, T.M.; An, W. Detecting non-hardhat-use by a deep learning method from far-field surveillance videos. *Autom. Constr.* **2018**, *85*, 1–9. [CrossRef]
7. Liu, W.; Anguelov, D.; Erhan, D.; Szegedy, C.; Reed, S.; Fu, C.Y.; Berg, A.C. Ssd: Single shot multibox detector. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016*; Proceedings, Part I; Springer: Cham, Switzerland, 2016; pp. 21–37.

8. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788.
9. Redmon, J.; Farhadi, A. YOLO9000: Better, faster, stronger. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, USA, 21–26 July 2017; pp. 7263–7271.
10. Redmon, J.; Farhadi, A. Yolov3: An incremental improvement. *arXiv* **2018**, arXiv:1804.02767. [CrossRef]
11. He, C.; Tan, S.; Zhao, J.; Ergu, D.; Liu, F.; Ma, B.; Li, J. Efficient and Lightweight Neural Network for Hard Hat Detection. *Electronics* **2024**, *13*, 2507. [CrossRef]
12. Liang, H.; Seo, S. Automatic detection of construction workers' helmet wear based on lightweight deep learning. *Appl. Sci.* **2022**, *12*, 10369. [CrossRef]
13. Xing, J.; Zhan, C.; Ma, J.; Chao, Z.; Liu, Y. Lightweight detection model for safe wear at worksites using GPD-YOLOv8 algorithm. *Sci. Rep.* **2025**, *15*, 1227. [CrossRef] [PubMed]
14. Fan, Z.; Wu, Y.; Liu, W.; Chen, M.; Qiu, Z. Lg-yolov8: A lightweight safety helmet detection algorithm combined with feature enhancement. *Appl. Sci.* **2024**, *14*, 10141. [CrossRef]
15. Bochkovskiy, A.; Wang, C.Y.; Liao, H.Y.M. Yolov4: Optimal speed and accuracy of object detection. *arXiv* **2020**, arXiv:2004.10934. [CrossRef]
16. Zhao, Z.Q.; Zheng, P.; Xu, S.t.; Wu, X. Object detection with deep learning: A review. *IEEE Trans. Neural Netw. Learn. Syst.* **2019**, *30*, 3212–3232. [CrossRef] [PubMed]
17. Hao, W.; Zhili, S. Improved mosaic: Algorithms for more complex images. *J. Phys. Conf. Ser.* **2020**, *1684*, 012094. [CrossRef]
18. Zhang, T.; Wang, F.; Wang, W.; Zhao, Q.; Ning, W.; Wu, H. Research on fire smoke detection algorithm based on improved YOLOv8. *IEEE Access* **2024**, *12*, 117354–117362. [CrossRef]
19. Li, C.; Yao, A. KernelWarehouse: Rethinking the design of dynamic convolution. *arXiv* **2024**, arXiv:2406.07879. [CrossRef]
20. Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the International Conference on Machine Learning, Lille, France, 6–11 July 2015; pp. 448–456.
21. Wu, Y.; He, K. Group normalization. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 3–19.
22. Zheng, Z.; Wang, P.; Liu, W.; Li, J.; Ye, R.; Ren, D. Distance-IoU loss: Faster and better learning for bounding box regression. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 12993–13000.
23. Ma, S.; Xu, Y. Mpdiou: A loss for efficient and accurate bounding box regression. *arXiv* **2023**, arXiv:2307.07662. [CrossRef]
24. Tong, Z.; Chen, Y.; Xu, Z.; Yu, R. Wise-IoU: Bounding box regression loss with dynamic focusing mechanism. *arXiv* **2023**, arXiv:2301.10051.
25. Zhang, C.; Du, Y.; Xiao, X.; Gu, Y. GE-YOLO: A lightweight object detection method for helmet wearing state. *Signal Image Video Process.* **2025**, *19*, 411. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.